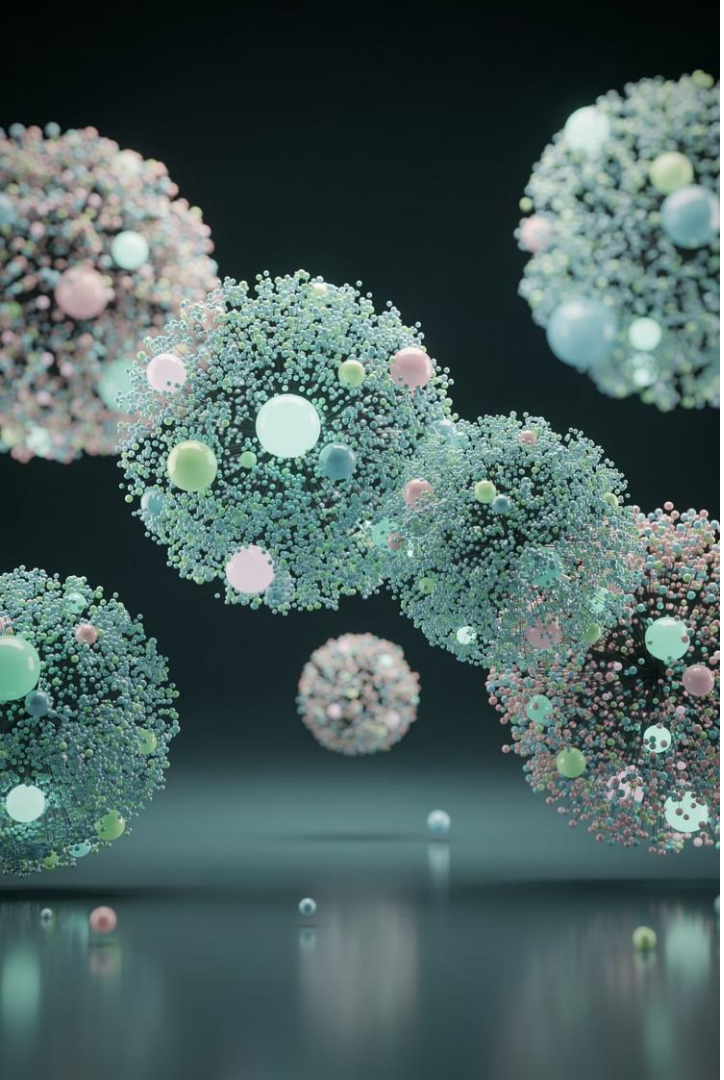


תל-חי
אוניברסיטת
קריית שמונה בגליל

Unsupervised Learning and Clustering

Dr. Marcelo Feighelstein

Learning Objectives

By the end of this lecture, you should be able to confidently answer each of the following questions:

1

Unsupervised Learning

What makes it different from supervised learning?

2

Clustering

What does it mean to "cluster" data?

3

K-Means

How does the algorithm group similar samples?

4

Silhouette Score

How do we evaluate whether our clustering is good?

5

Hierarchical Clustering

What is a dendrogram and why does it matter?

6

Distance & Linkage

How do metric and linkage choices affect results?

Supervised vs. Unsupervised Learning

Supervised Learning

You have labeled data — both features **X** and target labels **y**. The goal is to learn a mapping from inputs to outputs and predict labels for new, unseen samples.

- Training data: **(X, y)**
- Task: predict y for new X
- Examples: classification, regression
- Evaluation: accuracy, RMSE, F1

Unsupervised Learning

You only have features **X** — no labels at all. The goal is to discover hidden patterns, groupings, or structure within the data itself, without any prior guidance.

- Training data: **X only**
- Task: discover structure
- Examples: clustering, dimensionality reduction
- Evaluation: much harder — no ground truth

Think of supervised learning as studying with an answer key. Unsupervised learning is exploring a dataset with no answer key at all.



What Is Clustering?

Clustering is the task of **grouping similar samples together** so that points within a cluster are more alike than points in different clusters — all without using labels.

Intra-Cluster

Points **inside** the same cluster should be close to each other — high similarity within groups.

Inter-Cluster

Points in **different** clusters should be far apart — low similarity across groups.

No Labels Required

Clustering works on raw features only. Labels, if available, are set aside and never used during training.

Why Is Clustering Useful?

Customer Segmentation

Group shoppers by purchasing behavior to personalize marketing campaigns and product recommendations.

Patient Subtypes

Discover clinically distinct patient groups from biological measurements to guide treatment decisions.

Document Organization

Automatically group news articles, papers, or emails by topic without manual tagging.

Sensor Data Patterns

Detect anomalies or recurring patterns in IoT or industrial sensor streams.

Exploratory Analysis

Understand dataset structure before applying supervised learning — clustering can guide feature engineering.

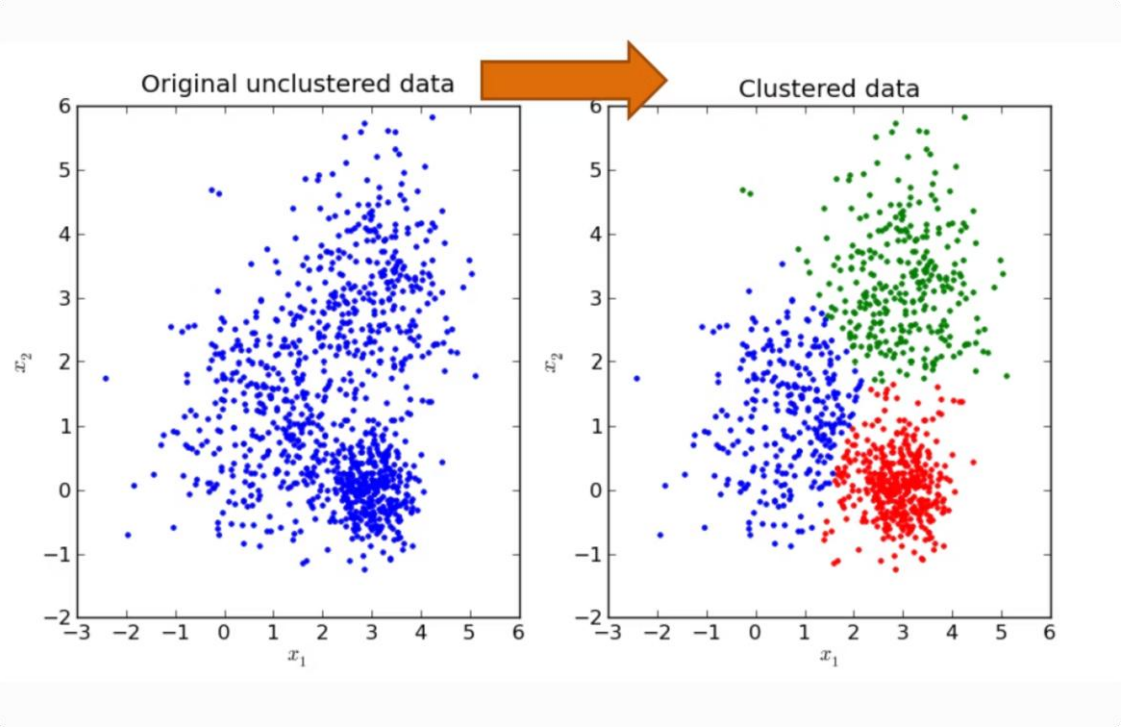
⚠ Important: Clustering can reveal hidden structure, but groups are not automatically meaningful. Always interpret results carefully in context.

What Is Clustering? A First Look

The Idea

- We start with **unlabeled data** — a cloud of points with no colors, no categories
- We choose a number of clusters K (e.g. K=3) and apply a **clustering algorithm**
- The result: each point is assigned to a group — **CLUSTERING RESULT**

The algorithm discovers structure without being told what to look for



The Three Core Challenges

Unsupervised learning sounds powerful — but it comes with unique difficulties that supervised learning does not face:

How does the cluster magic work?

The algorithm mechanism.

Which algorithm to use? What distance metric? Every choice shapes the result significantly.

How do we know the number of clusters K?

Choosing K.

Data often has many features — how do we visualize structure? Techniques like PCA or t-SNE help project high-dimensional data into 2D.

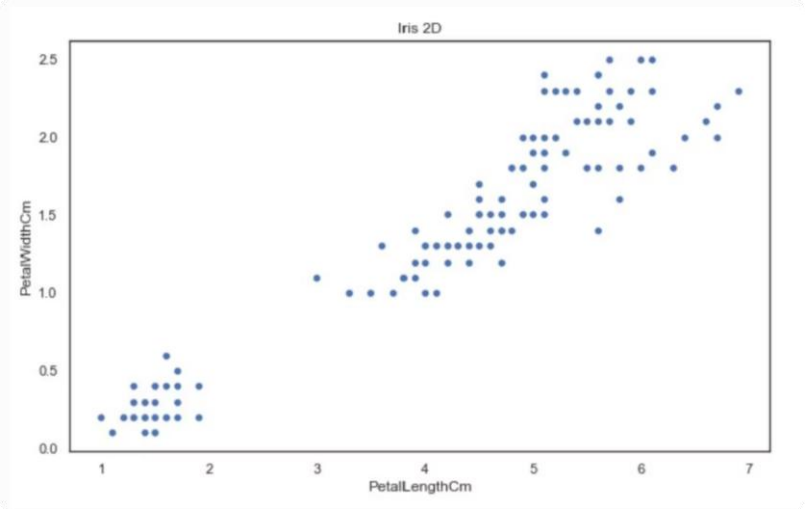
How do we measure if a clustering is good?

Evaluation metrics.

Without labels, how do we know if our clustering is good? We need special metrics like the silhouette score.

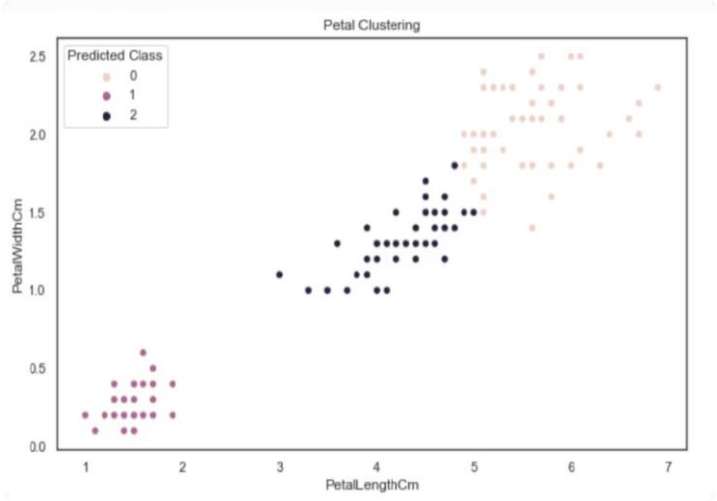
Clustering Example: Iris Dataset

LABELED



Three known species colored by ground-truth label: *Iris-setosa*, *Iris-versicolor*, and *Iris-virginica*.

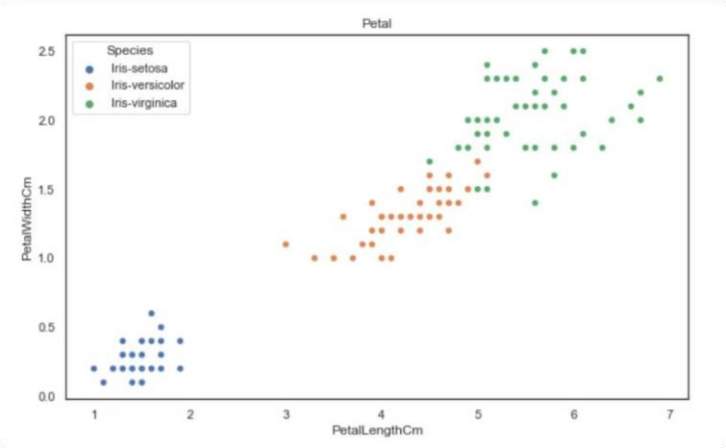
CLUSTERED KMEANS K=3



K-Means assigns each point to cluster **0**, **1**, or **2** — labels are arbitrary, determined purely by distance.

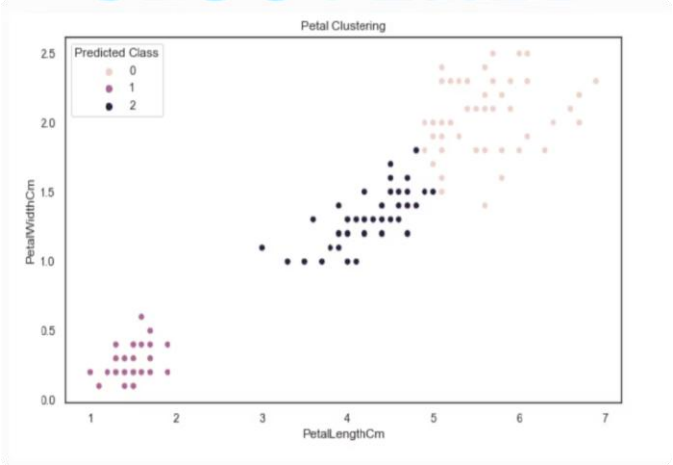
Labeled vs. Clustered: Iris Dataset

GROUND TRUTH (Labeled)



Three known species colored by ground-truth label: Iris-setosa (blue), Iris-versicolor (orange), and Iris-virginica (green).

K-MEANS CLUSTERING (K=3)



K-Means assigns each point to cluster 0, 1, or 2 — labels are arbitrary, determined purely by distance.

K-Means recovers the three species groups with high accuracy — but cluster labels (0, 1, 2) don't map directly to species names. The algorithm finds the structure without ever seeing the labels.



Clustering Example: The Iris Dataset

The classic Iris dataset contains **150 samples** from three flower species: *Setosa*, *Versicolor*, and *Virginica*. Each sample has four measured features.

Supervised Mode

- Use species labels as **y**
- Train a classifier on (X, y)
- Predict species for new flowers

Clustering Mode

- Temporarily **ignore** species labels
- Cluster only using measurements
- Compare discovered clusters to known species afterward as a sanity check

This makes Iris a perfect teaching dataset — we can evaluate how well clustering recovers the known biological groups.

K-Means: The Basic Idea

K-means is one of the most popular and intuitive clustering algorithms. Its goal is simple: divide the dataset into **K distinct groups**, where each group is represented by its **centroid** — the average position of all points in that cluster.

K = Your Choice

You decide how many clusters to create before running the algorithm. This is the key hyperparameter.

Centroid = Center

Each cluster has a centroid — the mean of all points assigned to it. Points are assigned to their nearest centroid.

Iterative Refinement

The algorithm alternates between assigning points and updating centroids until the assignments stabilize.

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=3, random_state=42).fit(X)
```

K-Means Intuitive Explanation

A fundamental aspect of K-Means is defining the number of clusters, denoted by **K**, before the algorithm begins its work.

Hyperparameter K

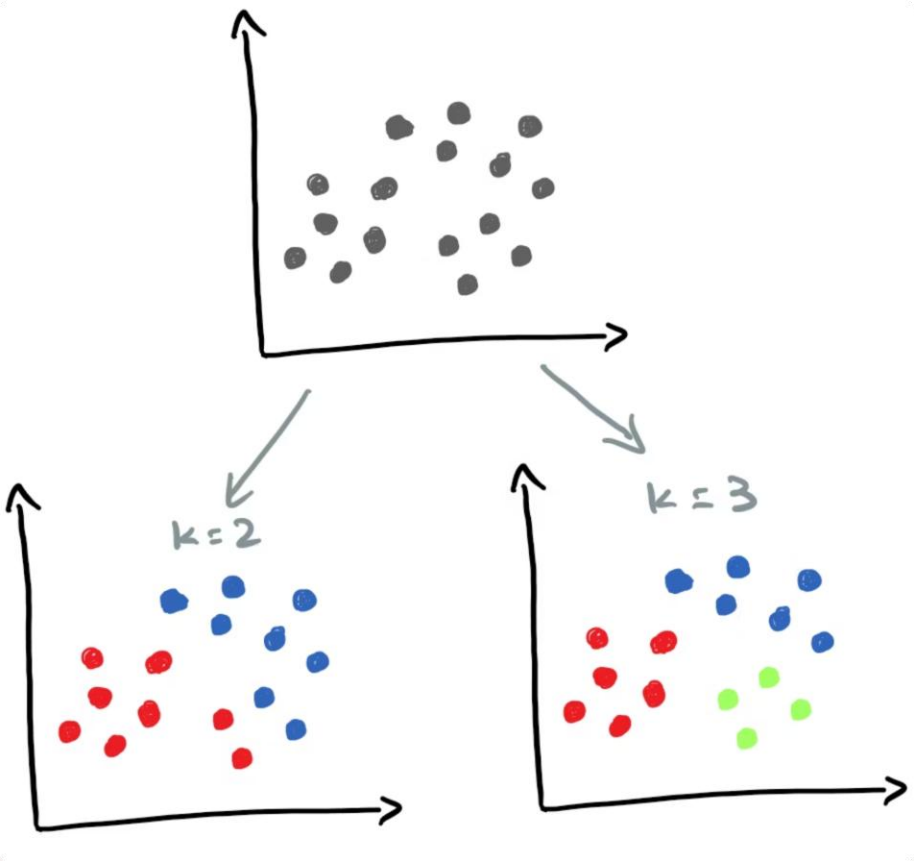
K is a hyperparameter that you must choose. It determines how many distinct groups the K-Means algorithm will form from your dataset.

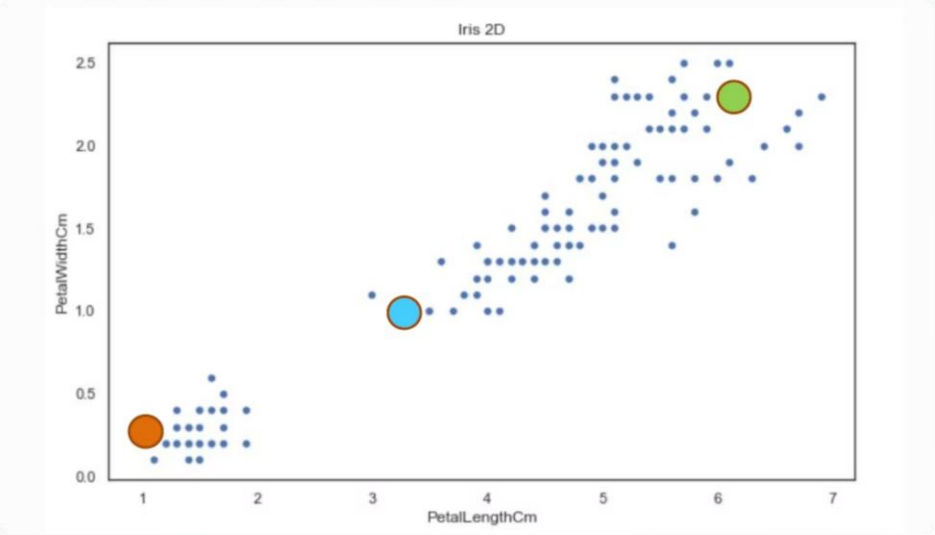
If K=3 we want to create 3 clusters

By setting K to 3, you instruct the algorithm to partition the data into three separate, coherent clusters.

If K=5 we want to create 5 clusters

Conversely, if you specify K=5, K-Means will endeavor to identify five distinct clusters within the data.



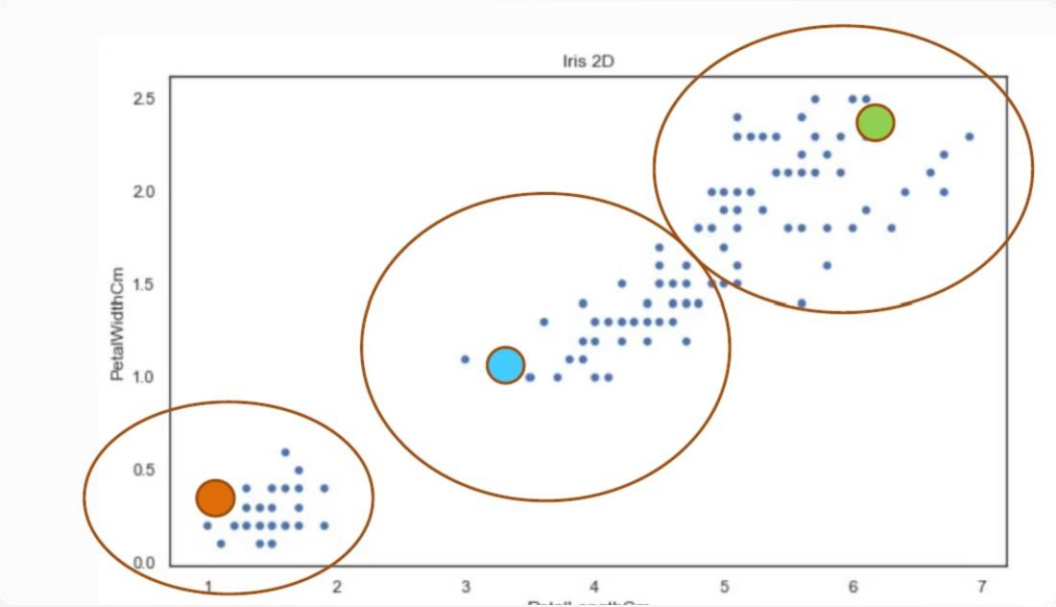


K-Means Intuitive Explanation

First Step — Random Centroids

Hyperparameter K = 3

Choose K=3 points randomly as initial centroids



K-Means Intuitive Explanation

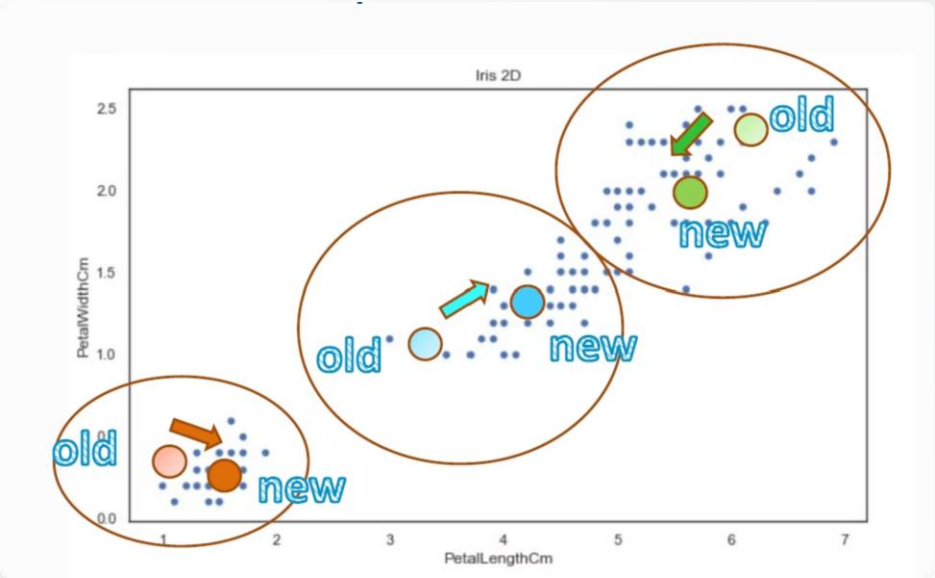
Second Step — Find Closest Centroid Neighbors

Hyperparameter K = 3

Choose K=3 points randomly

Each data point is assigned to the nearest centroid.

Three oval/ellipse regions are drawn around each centroid grouping the closest points together.



K-Means Intuitive Explanation

Third Step — Find Clusters Mass Center

- Calculate New centroids using Mean Coordinates of all points on each cluster
- Old centroids move to new positions (the mean of all points in each cluster)

Show "old" and "new" centroid positions with arrows indicating the shift

K-Means Intuitive Explanation

Calculate New Centroids

Calculate new centroids using Mean Coordinates of all points on each cluster.

Go to Step 2

Reassign each point to the nearest new centroid.

Points changing cluster — some points switch from one cluster to another as centroids move.

Repeat until Convergence

The algorithm iterates, moving centroids and reassigning points, until cluster assignments no longer change significantly.

K-Means Intuitive Explanation

STOP CONDITION

We stop when:

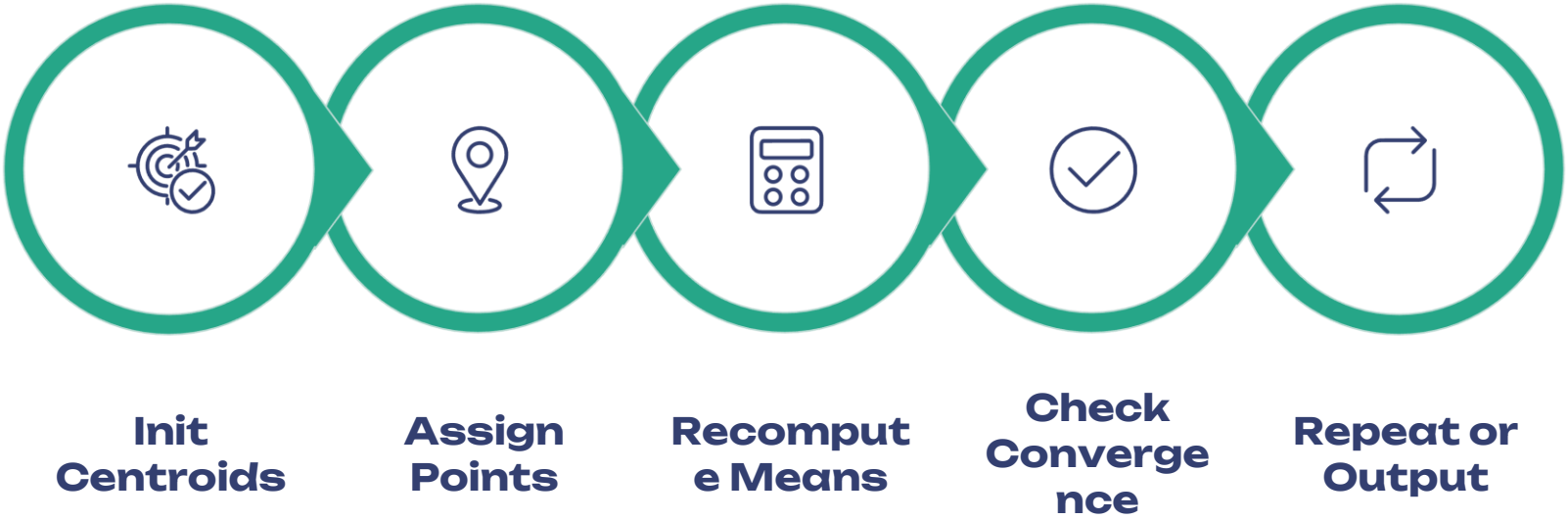
No Changes on Cluster

The algorithm stops when there are no more changes in cluster assignments for any data point.

Max Iterations Reached

The algorithm also stops if it reaches the maximum number of iterations, typically set to 300 by default.

K-Means: Step by Step



K-means is guaranteed to converge — assignments will eventually stop changing. However, the result depends on the initial centroid placement, so it is common practice to run the algorithm multiple times with different random seeds and keep the best result. In scikit-learn, this is controlled with the `n_init` parameter.

Choosing K: Why It Matters

K is the single most important decision you make before running K-means. Different values of K produce fundamentally different partitions of the data — and there is no one-size-fits-all answer.

1	2	3
K = 2	**K = 3**	**K = 4+**
The data is split into just two broad groups. Fine details and sub-clusters may be lost — some natural groupings get merged together.	Often a good fit for naturally three-cluster datasets like Iris. With the right K, clusters align well with real structure in the data.	Too many clusters can over-fragment the data — splitting natural groups into artificial sub-clusters that have no meaningful interpretation.

Choosing K is one of the **main challenges** of K-means. We'll use the silhouette score to guide this decision systematically.

K-Means: Practical Application (K=3)

Let's put the K-Means algorithm into practice using a simplified version of our Iris dataset. We'll set **K=3** directly, as we know there are three species in the original dataset. This example demonstrates how to initialize and fit the model.

```
from sklearn.cluster import KMeans
import numpy as np

# Sample data (e.g., first two features of Iris for visualization)
X = np.array([[5.1, 3.5], [4.9, 3.0], [6.3, 3.3],
              [5.0, 3.4], [6.0, 2.7], [5.5, 2.3],
              [6.8, 3.0], [5.7, 2.8], [6.7, 3.1]])

# Initialize K-Means with 3 clusters
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

# Fit the model to the data
kmeans.fit(X)

# Get cluster labels for each data point
labels = kmeans.labels_

# Get the coordinates of the cluster centroids
centroids = kmeans.cluster_centers_

print("Cluster Labels:", labels)
print("Centroids:", centroids)
```

The `n_clusters=3` parameter explicitly tells the algorithm to find three groups. The `random_state` ensures reproducibility of the initial centroid placement, and `n_init=10` runs the algorithm 10 times with different centroid seeds and picks the best result to avoid suboptimal local minima.

Distance and Similarity

Clustering fundamentally depends on how we **measure similarity** between data points. The choice of distance metric shapes which points end up in the same cluster.



Euclidean Distance

The straight-line distance between two points in feature space. Most commonly used. Works best when all features are on similar scales. This is the K-means default.



Manhattan Distance

Sum of absolute differences along each axis — like navigating city blocks. More robust to outliers than Euclidean. Also called L1 distance.



Correlation Distance

Measures similarity in **shape or pattern** rather than absolute values. Useful in genomics or time series where relative trends matter more than magnitude.

⚠ Always **scale your features** (e.g., StandardScaler) before computing distances. Features with larger ranges will dominate distance calculations otherwise.

Evaluating Clustering Quality

What Makes a Good Cluster?

Without ground-truth labels, we need internal metrics to judge cluster quality. A well-formed clustering has two key properties:

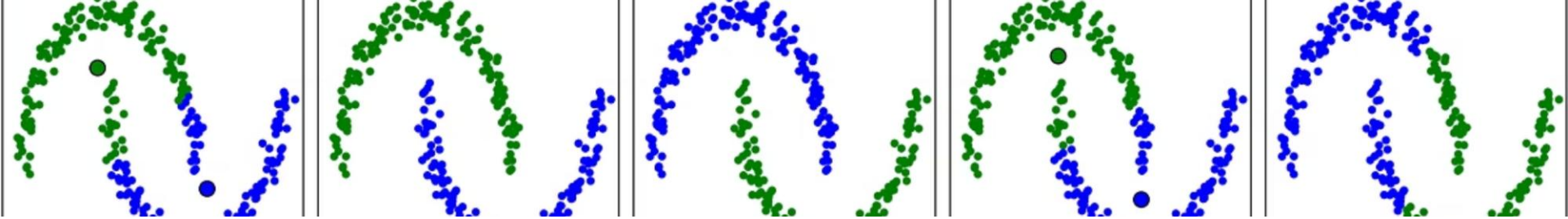
High Intra-Cluster Similarity

Points inside the same cluster are tightly packed and close to each other — low within-cluster variance.

Low Inter-Cluster Similarity

Points in different clusters are far apart — clusters are clearly separated from one another.

Evaluation quality depends heavily on your feature representation and chosen distance metric. Bad features → bad clusters, even with a great algorithm.



Choosing K: The Elbow Method

Determining the optimal number of clusters (**K**) for K-Means is crucial. The Elbow Method is a widely used heuristic to guide this decision by evaluating clustering performance across different values of K.

KEY METRIC

Understanding WCSS (Within-Cluster Sum of Squares)

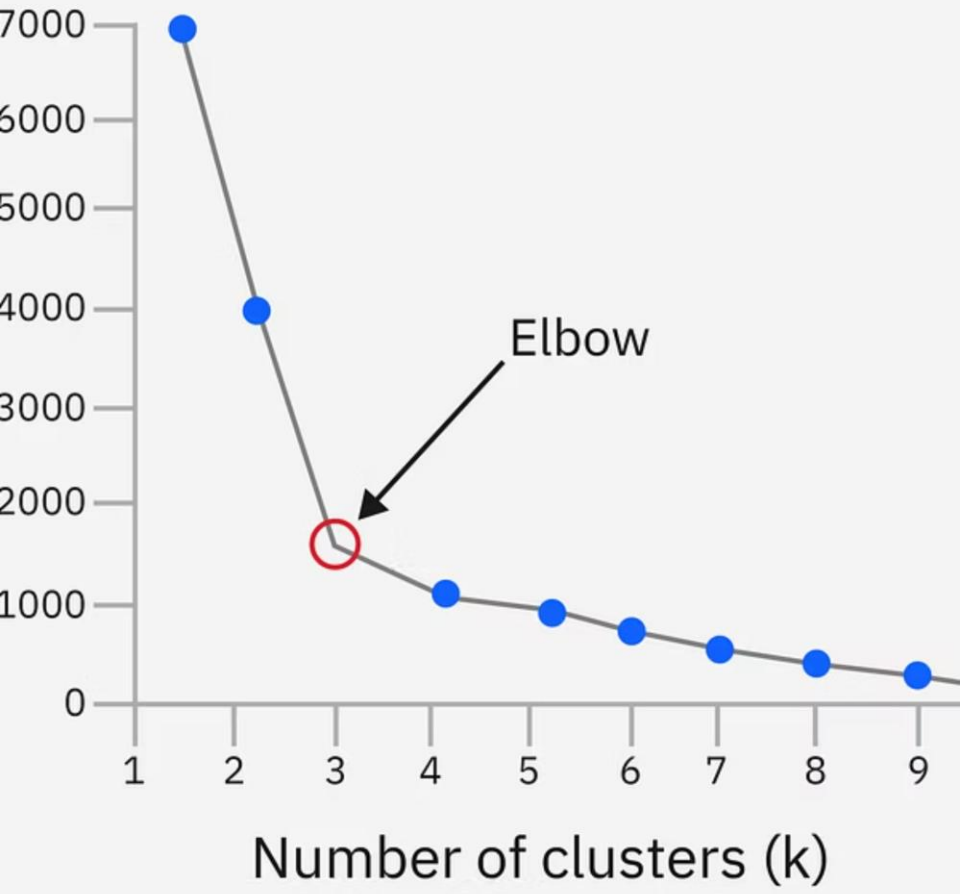
The Elbow Method relies on the **Within-Cluster Sum of Squares (WCSS)**, also known as inertia. WCSS measures the sum of the squared distances between each data point and the centroid of its assigned cluster. Essentially, it quantifies how compact and well-defined the clusters are.

Lower WCSS = Better Fit

A smaller WCSS value indicates that data points are closer to their respective cluster centroids, suggesting more compact and internally coherent clusters.

WCSS Decreases with K

As the number of clusters (K) increases, WCSS will naturally decrease or stay the same. This is because adding more centroids allows each point to be assigned to an even closer centroid, reducing the overall sum of squared distances.



Choosing K: Identifying the "Elbow" Point

To use the Elbow Method, we plot the WCSS values against different values of K. As K increases, WCSS decreases, but at some point, the rate of decrease will slow down significantly, forming an "elbow" shape in the graph.

The "elbow" point is considered the optimal K. Beyond this point, adding more clusters provides diminishing returns in terms of reducing WCSS, suggesting that the additional clusters are not adding substantial value or capturing more natural groupings.

The Elbow Method is a heuristic, not a definitive rule. Sometimes the "elbow" is not clear, requiring judgment or additional validation with other metrics.

The Silhouette Score

The **silhouette score** measures how well each point fits inside its assigned cluster, compared to how close it is to the nearest other cluster. It ranges from **−1 to +1**.

Close to +1

The point is well inside its cluster and far from others. **Excellent** clustering result.

Around 0

The point sits near the boundary between clusters. Clusters may be **overlapping** or poorly separated.

Negative

The point may have been assigned to the **wrong cluster** — it is closer to a neighboring cluster than its own.

The Formula

$$s(i) = \frac{b(i) - a(i)}{\max(a(i),\ b(i))}$$

a(i) — Intra-cluster cohesion

Mean distance from point *i* to all other points in its **own cluster**. Measures how well the point fits within its cluster. **Lower is better.**

b(i) — Inter-cluster separation

Mean distance from point *i* to all points in the **nearest neighboring cluster**. Measures how far the point is from the next best cluster. **Higher is better.**

s(i) — Silhouette score for point *i*

The resulting silhouette score, ranging from **−1 to +1**. The overall silhouette score is the **mean of s(i)** across all points.

```
from sklearn.metrics import silhouette_score
score = silhouette_score(X, km.labels_)
```

Hyperparameter Tuning for K

Finding the Best K

Since we can't rely on labels to evaluate clustering, we run K-means for a range of K values and compute the silhouette score for each. The K with the **highest silhouette score** is the best candidate — but always verify it makes domain sense.

01

Choose a range of K

For example, K = 2 through 10

02

Fit & Score

Run KMeans and compute silhouette_score for each K

03

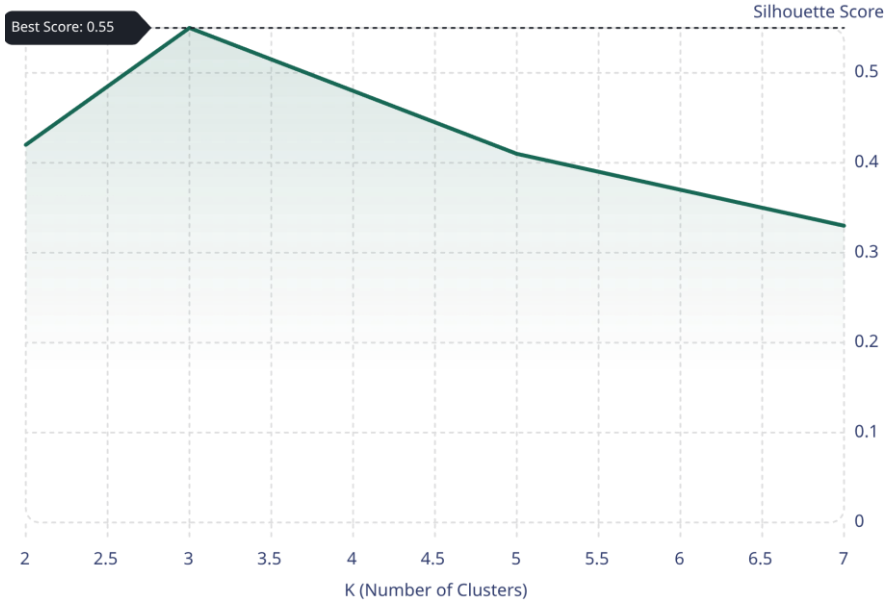
Plot & Inspect

Find the peak silhouette score on a line plot

04

Select K

Pick the best K and interpret the clusters in context



In this example, K=3 yields the highest silhouette score — a strong signal that three clusters best describe this dataset.

Finding Optimal K: Python Implementation

We can systematically determine the best number of clusters (K) for our dataset by iterating through a range of K values and calculating the Silhouette Score for each. The K that yields the highest Silhouette Score is often considered the optimal choice.

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import numpy as np

# Assume 'X' is your preprocessed feature dataset
# Example: X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)

range_n_clusters = list(range(2, 11)) # Test K from 2 to 10
silhouette_avg_scores = []

for num_clusters in range_n_clusters:
    # Initialize KMeans with the current number of clusters
    kmeans = KMeans(n_clusters=num_clusters, random_state=42, n_init=10)
    cluster_labels = kmeans.fit_predict(X)

    # Compute the silhouette score for the current clustering
    silhouette_avg = silhouette_score(X, cluster_labels)
    silhouette_avg_scores.append(silhouette_avg)
    print(f"K = {num_clusters}: Silhouette Score = {silhouette_avg:.4f}")

# Find the K value corresponding to the highest silhouette score
optimal_k_index = np.argmax(silhouette_avg_scores)
optimal_k = range_n_clusters[optimal_k_index]

print(f"\nOptimal K based on Silhouette Score: {optimal_k}")

# You would typically plot silhouette_avg_scores against range_n_clusters
# to visualize the peak, similar to the graph on the previous slide.
```

This code iterates through possible K values, runs K-Means, and computes the silhouette score. The K value with the highest score indicates the most appropriate number of clusters, suggesting well-separated and dense clusters.



Limitations of K-Means

K-means is powerful but not universal. Understanding its weaknesses helps you know when to use it — and when to try something else.

Must Choose K in Advance

You must specify K before running. If the true number of clusters is unknown, trial-and-error with silhouette scoring is required.

Sensitive to Feature Scaling

Features with larger numeric ranges dominate the distance computation. Always apply `StandardScaler` before K-means.

Assumes Spherical Clusters

K-means struggles with elongated, crescent-shaped, or ring-shaped clusters. It forces every cluster into a roughly circular shape.

Sensitive to Outliers

Outliers can pull centroids away from the true cluster center, distorting the entire solution. Consider robust preprocessing.

Hierarchical Clustering

A Different Approach

Instead of assigning all points to K clusters at once, **hierarchical clustering** builds a full **tree of nested clusters** — called a **dendrogram**. This tree shows how every sample relates to every other sample at multiple levels of granularity.

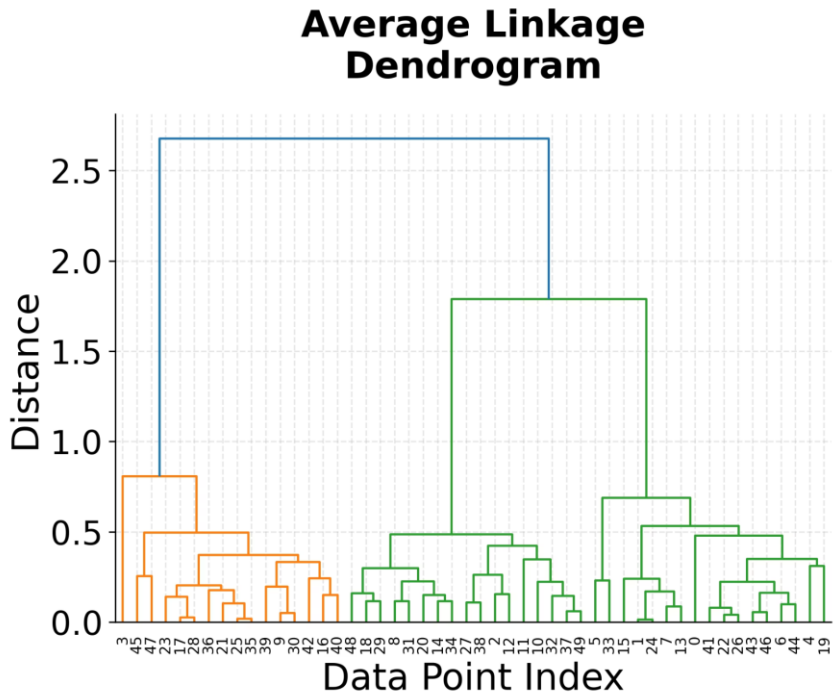
No K Required Up Front

You choose the number of clusters *after* building the full hierarchy, by cutting the dendrogram at a chosen height.

Reveals Nested Structure

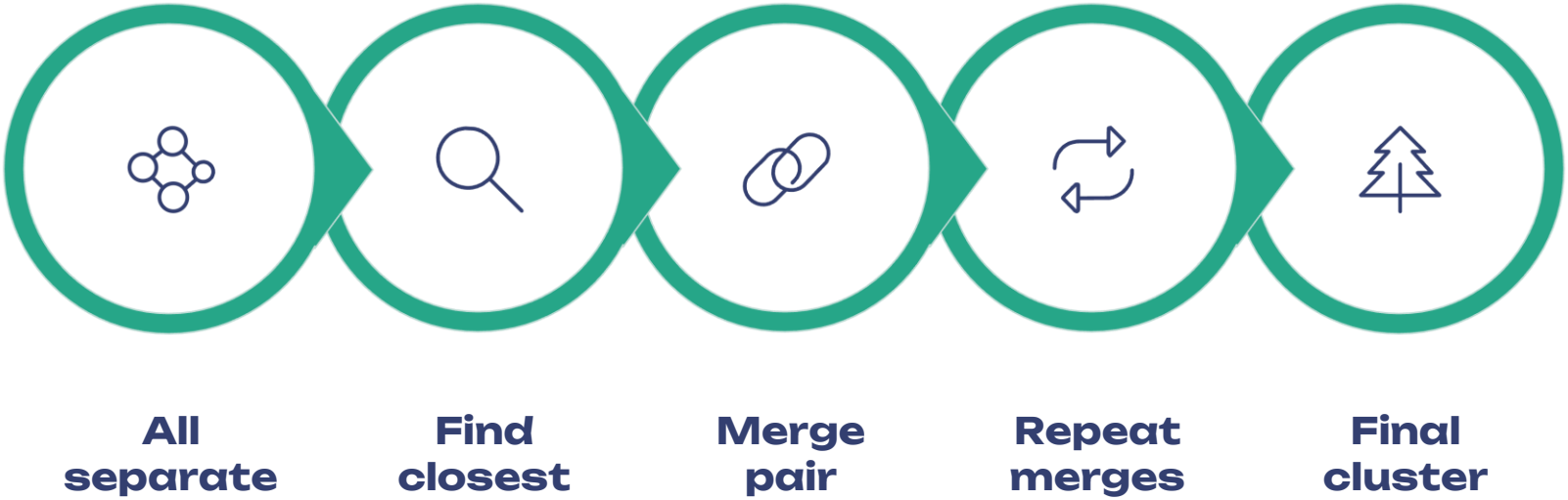
The tree shows sub-clusters within clusters — useful for understanding data at multiple scales of similarity.

```
from sklearn.cluster import AgglomerativeClustering
```



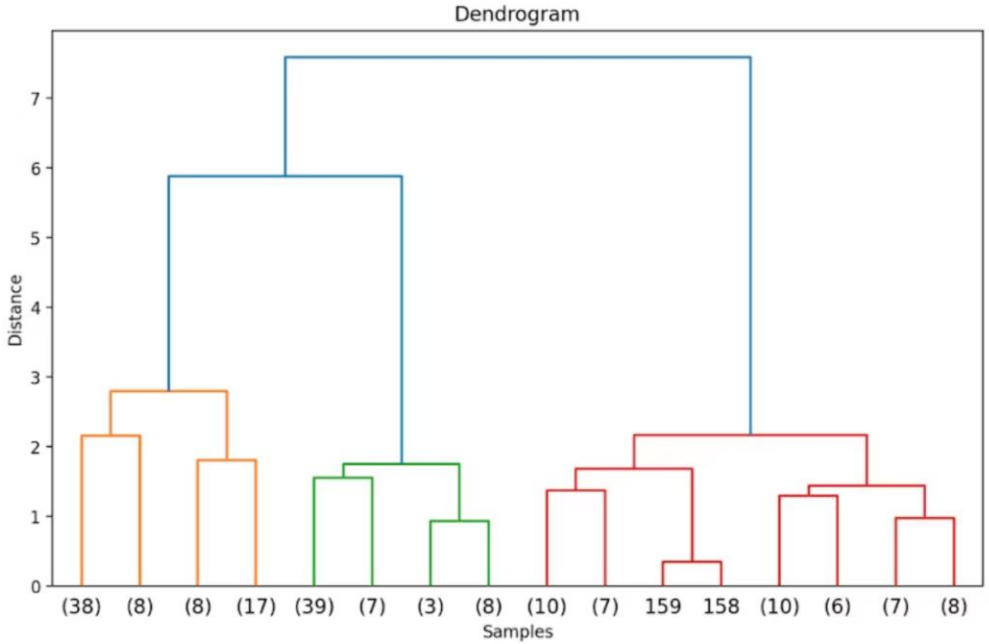
Agglomerative Clustering: Bottom-Up

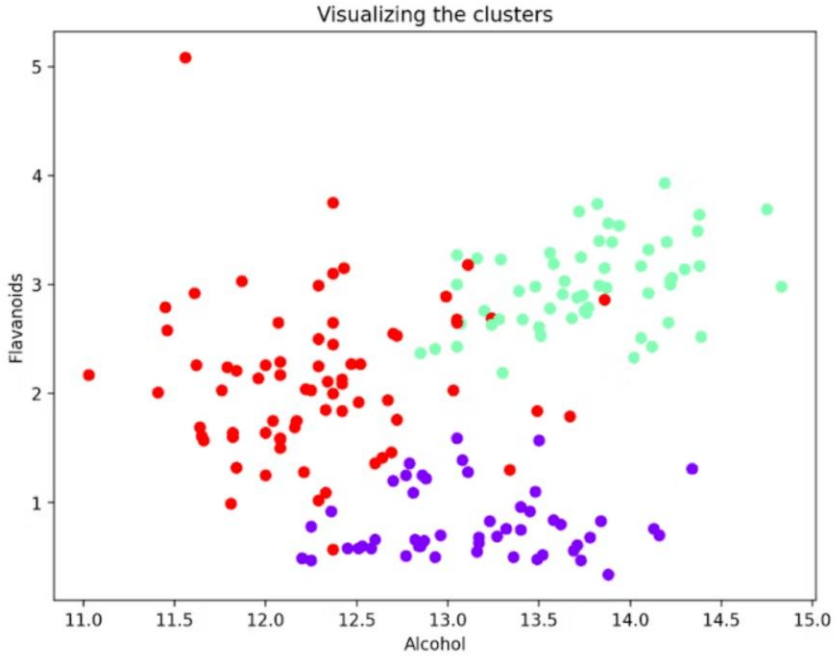
The most common approach to hierarchical clustering is **agglomerative** (bottom-up): start small and merge upward.



The final single cluster containing all samples is not the useful output — the value lies in the **full hierarchy**. By cutting the dendrogram at a chosen height, you extract any number of clusters you want. Use `scipy.cluster.hierarchy.dendrogram` to visualize the tree.

Intro to Hierarchical Clustering





Cutting the Dendrogram: Defining Clusters

You don't need to specify K upfront — just "cut" the dendrogram at a chosen height to define your clusters.

Visual Inspection for Natural Breaks

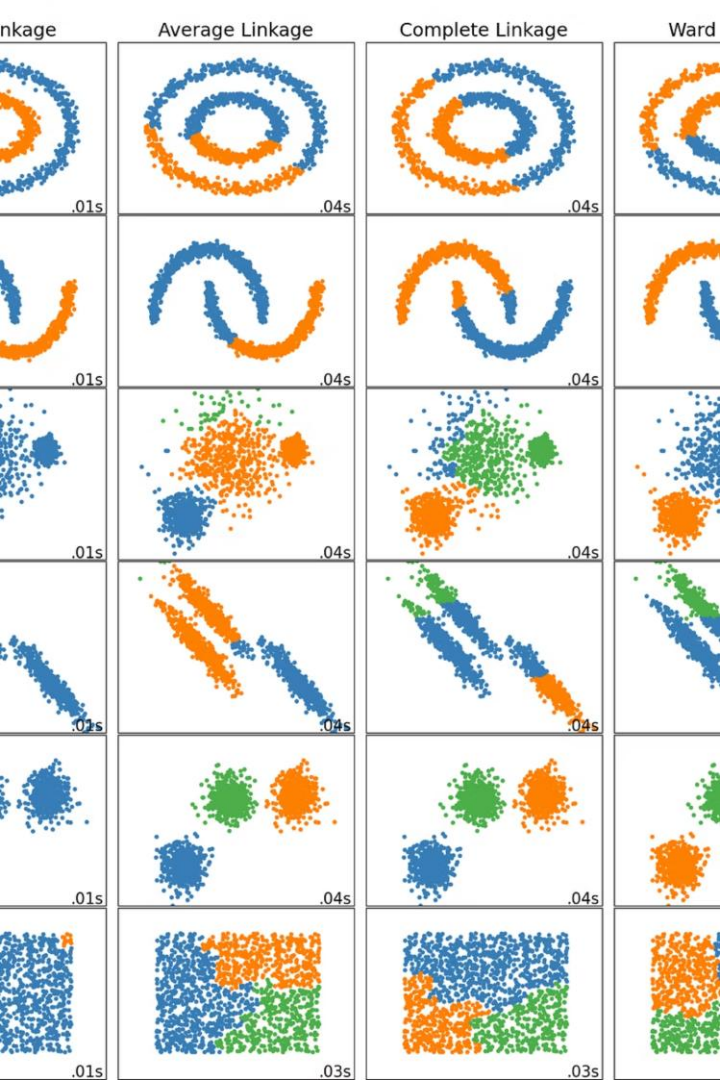
Cut through large gaps (long vertical lines) to find natural cluster boundaries.

Targeting a Specific *K*

Draw a horizontal line that intersects exactly K segments to produce a desired number of clusters.

Leveraging Domain Expertise

Choose the cut that makes the most sense given your data's real-world context.



Linkage Methods

When merging clusters, hierarchical clustering needs a rule to measure the **distance between two clusters** (not just two points). This rule is called the **linkage method**, and it dramatically changes the shape of the resulting dendrogram and clusters.

What changes?

- Shape of clusters (elongated vs. compact)
- Sensitivity to noise and outliers
- Dendrogram structure

4 Common Methods:

- Single Linkage
- Complete Linkage
- Average Linkage
- Ward Linkage (recommended)

Single Linkage

Distance = closest pair of points between two clusters

Imagine two groups of people — single linkage measures the distance between the two people who are closest to each other (one from each group).

How it works

Merge the two clusters whose nearest members are closest. d(A,B) = min distance between any point in A and any point in B.

Chaining Effect

Tends to create long, snake-like clusters. One outlier can "bridge" two groups together, causing them to merge prematurely.

Best for

Detecting elongated or irregular-shaped clusters. Poor for compact, globular clusters.

Weakness

Very sensitive to noise and outliers.

Complete Linkage

Distance = farthest pair of points between two clusters

Imagine two groups of people — complete linkage measures the distance between the two people who are farthest apart (one from each group). Both groups must be "fully" close to merge.

How it works

d(A,B) = max distance between any point in A and any point in B. Clusters only merge when ALL their members are close enough.

Compact Clusters

Produces tight, roughly equal-sized clusters. Avoids the chaining problem of single linkage.

Best for

Compact, globular clusters where you want balanced group sizes.

Weakness

Can break large clusters apart; sensitive to outliers at cluster boundaries.

Average Linkage

Distance = average of ALL pairwise distances between two clusters

Instead of looking at just the closest or farthest pair, average linkage considers every possible pair of points (one from each group) and takes the average distance.

How it works

d(A,B) = average of all distances between every point in A and every point in B. A balanced middle ground.

Balanced Result

Less extreme than single or complete linkage. Produces moderately compact clusters without severe chaining.

Best for

General-purpose use when you're unsure which linkage to pick. More robust than single linkage.

Weakness

Computationally more expensive; can still be affected by outliers.

Ward Linkage ⭐ Recommended

Merge clusters that minimize the increase in total within-cluster variance

Instead of measuring distance between points, Ward asks: "if I merge these two clusters, how much more spread out (variance) will the combined cluster be?" It always picks the merge that increases variance the least.

How it works

At each step, merge the pair of clusters that results in the smallest increase in total within-cluster sum of squares (WCSS). Minimizes intra-cluster variance.

Most Balanced

Tends to produce the most compact, equally-sized clusters. Often gives the most interpretable dendrograms.

Best for

Most real-world datasets. The default choice in scikit-learn's AgglomerativeClustering.

Limitation

Assumes clusters are roughly spherical and similar in size. Does not work with non-Euclidean distances.

```
from sklearn.cluster import AgglomerativeClustering
model = AgglomerativeClustering(n_clusters=3, linkage='ward')
labels = model.fit_predict(X)
```

Ward Linkage: Python Implementation

Let's put Ward linkage into practice using a synthetic dataset. This example demonstrates how to apply `AgglomerativeClustering` and visualize the resulting clusters on a scatter plot.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

# 1. Generate synthetic data
X, y = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)

# 2. Apply Agglomerative Clustering with Ward linkage
#    n_clusters can be set to None if you plan to cut the dendrogram later,
#    but here we pre-define it for direct clustering visualization.
model = AgglomerativeClustering(n_clusters=4, linkage='ward')
labels = model.fit_predict(X)

# 3. Visualize the clusters
plt.figure(figsize=(10, 6))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=50, alpha=0.8)
plt.title('Hierarchical Clustering with Ward Linkage')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.colorbar(label='Cluster Label')
plt.show()

# Optional: Visualize the dendrogram
# Z = linkage(X, 'ward')
# plt.figure(figsize=(12, 5))
# dendrogram(Z)
# plt.title('Dendrogram for Ward Linkage')
# plt.xlabel('Sample index')
# plt.ylabel('Distance')
# plt.show()
```

This code generates 300 data points forming 4 distinct blobs, then applies `AgglomerativeClustering` with the `'ward'` linkage method to identify 4 clusters. The output scatter plot visually confirms the model's ability to separate these clusters effectively.

Summary & Discussion

Unsupervised Learning

Discovers structure in data **without labels**. No ground truth to guide us — exploration is the goal.

K-Means

Simple, fast, and effective for compact clusters. Requires choosing K and is sensitive to scaling and initialization.

Silhouette Score

Evaluates clustering quality without labels. Use it to systematically compare different values of K.

Hierarchical Clustering

Builds a full dendrogram — no K required upfront. Linkage method and distance metric shape the result.

💬 Discussion Question

If a clustering algorithm finds distinct groups in a dataset, **how can we decide whether those groups are actually meaningful in the real world?** What additional evidence — beyond the algorithm's output — would you look for?

Key tools: `KMeans` · `silhouette_score` · `AgglomerativeClustering` · `scipy.cluster.hierarchy.dendrogram`

תל-חי
אוניברסיטת
קריית שמונה בגליל

Thank You!

Go explore data — get your hands dirty, break things, and learn by doing.

Email

MarcelFei@telhai.ac.il

Office Hours

Per Request

Location

Building 6, Room 309, Eastern Campus